Applications built on top of Kafka ***are event driven***. They typically take one or more data streams as the input and continuously transform these data into a new stream. The transformation often ***includes streaming operations such as filtering, projection, joining, and aggregation***. Expressing those operations in low-level Java code is inefficient and error prone. ***Kafka Streams*** provides a handful of high-level abstractions for developers to express those common streaming operations concisely and is a very powerful tool for building event-driven applications in Java.

****

Some typical examples of using event streaming applications:

Credit card fraud—A credit card owner may be unaware of unauthorized use. By reviewing purchases as they happen against established patterns (location, general spending habits), you may be able to detect a stolen credit card and alert the owner.

Intrusion detection—The ability to monitor aberrant behavior in real time is critical for the protection of sensitive data and the well-being of an organization.

The Internet of Things—With IoT, sensors are located in all kinds of places and send back data frequently. The ability to quickly capture and process this data meaningfully is essential; anything less diminishes the effect of deploying these sensors.

The financial industry—The ability to track market prices and direction in real time is essential for brokers and consumers to make effective decisions about when to sell or buy.

Sharing data in real-time—Large organizations, like corporations or conglomerates, that have many applications need to share data in a standard, accurate, and real-time way.

****

We'll define an event simply as "something that happens" like a ***customer making a purchase*** (*online or in-person*) or ***clicking a link on a web page*** or a ***sensor transmitting data***. Either people or machines can generate events. It's the sequence of events and the constant flow of them that make up an event stream.

****

Events conceptually contain three main components:

- ***Key***—An identifier for the event
- ***Value***—The event itself
- ***Timestamp***—When the event occurred

****

Table 1.1 Definitions

Event	Something that occurs and attributes about it recorded
Event Stream	A series of events captured in real-time from sources such as mobile or IoT devices
Event Streaming Platform	Software to handle event streams—capable of producing, consuming, processing, and storage of event streams
Apache Kafka	The premier event streaming platform—provides all the components of an event streaming platform in one battle-tested solution
Kafka Streams	The native event stream processing library for Kafka

****

The Kafka event streaming platform provides the core capabilities to implement your event streaming application from end to end. We can break down these capabilities into three main areas: *publishing/consuming, durable storage, and processing*. This *move, store, and process trilogy enables Kafka to operate as the central nervous system for your data*.

****

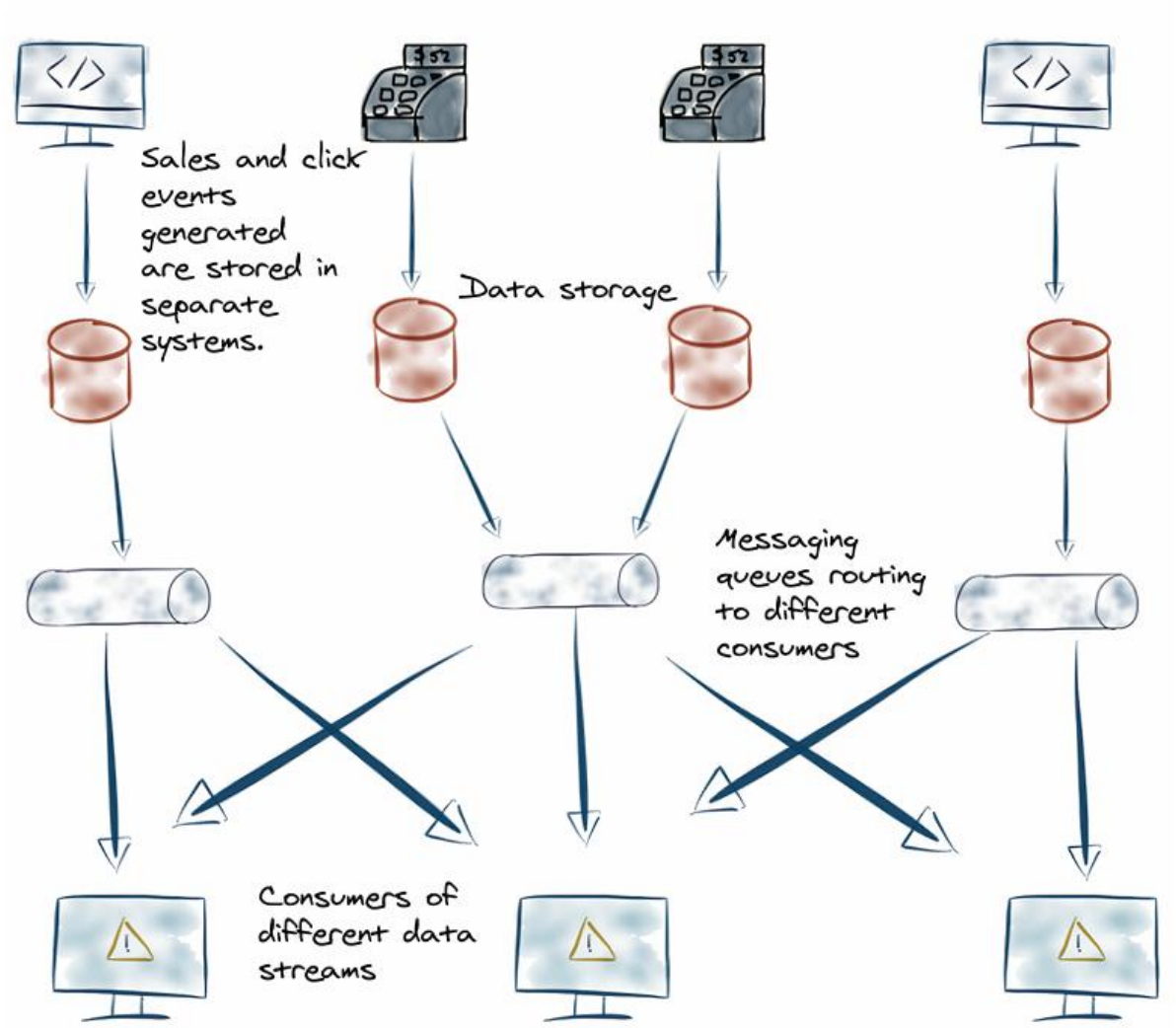

Figure 1.2 Initial event-streaming architecture leads to complexity as the different departments and data stream sources need to be aware of the other sources of events.

****

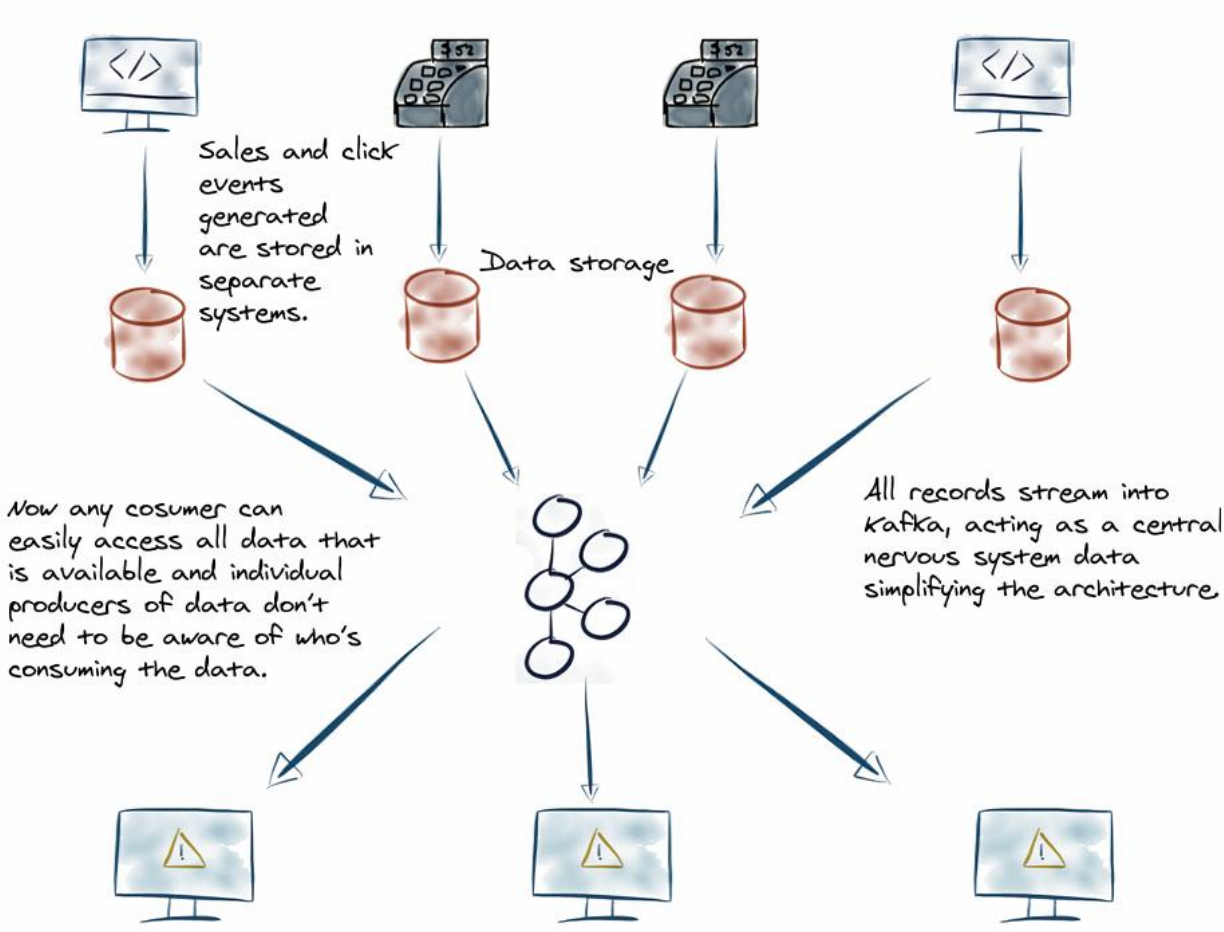

Figure 1.3 Using the Kafka event streaming platform with a simplified architecture

****

Adding the *Kafka event streaming platform* simplifies the architecture dramatically. All components now send their records to Kafka. Additionally, consumers read data from Kafka with no awareness of the producers. At a high level, Kafka is a distributed system of servers and clients. *The servers are called brokers*; **the clients are record producers sending records to the brokers**, and *the consumer clients read records for the processing of events*.

****

Kafka brokers

Kafka brokers durably store your records in contrast with traditional messaging systems (**RabbitMQ** or **ActiveMQ**), where the messages are ephemeral. The brokers store the data agnostically as the key-value pairs (and some other metadata fields) in byte format and are somewhat of a black box to the broker.

Preserving events has more profound implications concerning the difference between messages and events. You can think of messages as "tactical" communication between two machines, while events represent business-critical data you don't want to throw away.

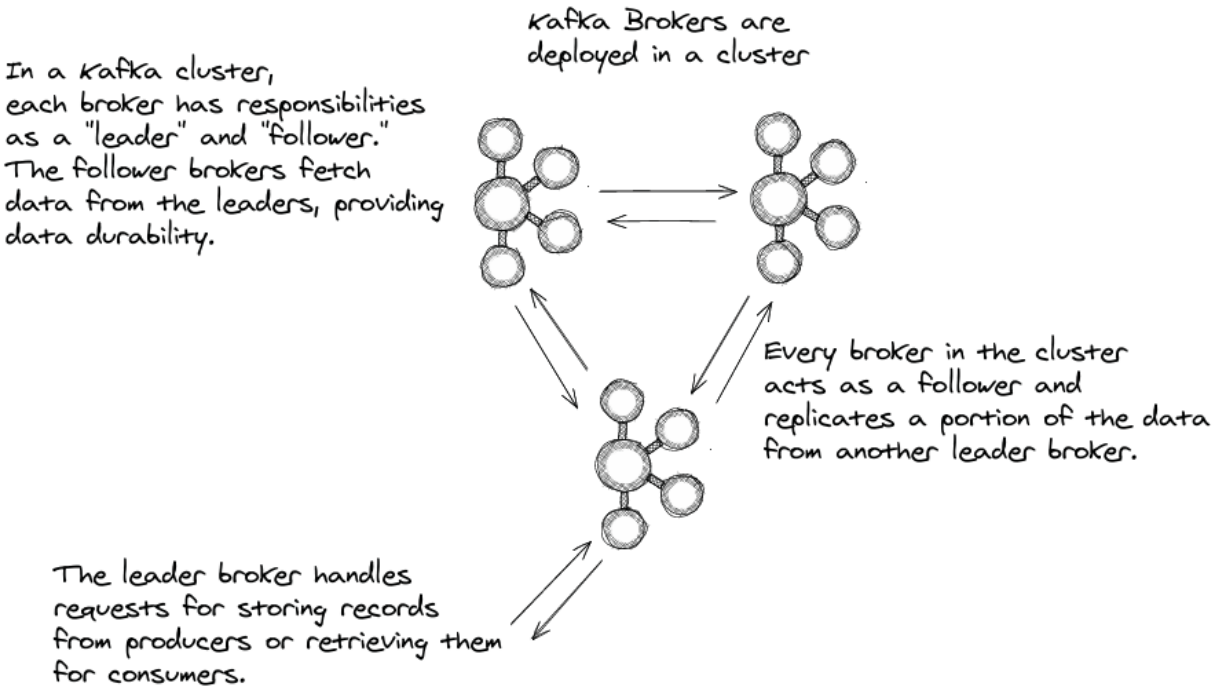

Figure 1.4 You deploy brokers in a cluster, and brokers replicate data for durable storage.

****

Schema Registry

Schema Registry stores schemas of the event records. Schemas enforce a contract for data between producers and consumers. Schema Registry also provides serializers and deserializers, supporting different tools that are Schema Registry aware. Providing (de)serializers means you don't have to write your serialization code.

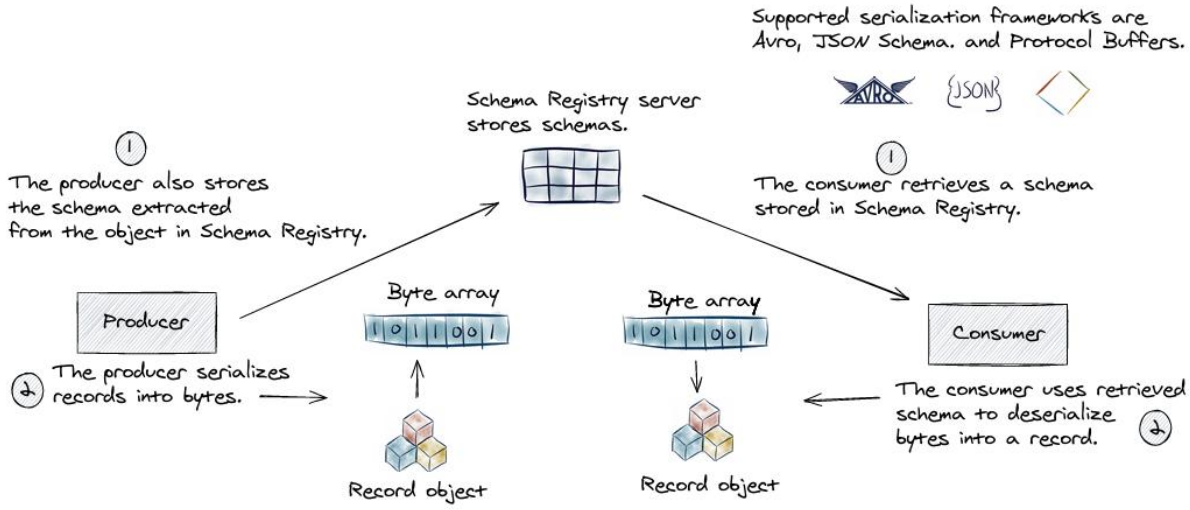

Figure 1.5 Schema registry enforces data modeling across the platform.

****

Kafka Connect

Kafka Connect provides an abstraction over the producer and consumer clients for importing data to and exporting data from Apache Kafka.

1. Kafka Connect is essential in connecting external data stores with Apache Kafka.
2. It also provides an opportunity to perform lightweight data transformations with *Simple Message Transforms (SMTs)* when exporting or importing data.

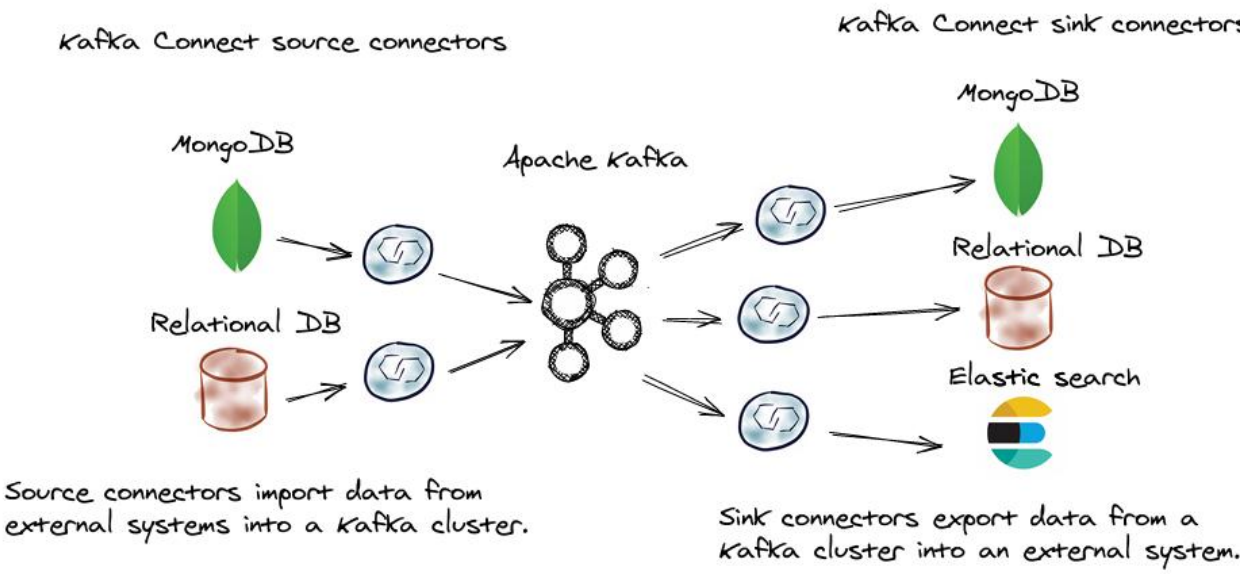

Figure 1.7 Kafka Connect bridges the gap between external systems and Apache Kafka.

****

Kafka Streams

Kafka Streams is Kafka's native stream processing library. Kafka Streams is written in the Java programming language and is used by client applications at the perimeter of a Kafka cluster; it is not run inside a Kafka broker. It supports performing operations on event data, including: *transformations and stateful operations like joins and aggregations*. Kafka Streams is where you'll do the heart of your work when dealing with events.

****

ksqlDB

ksqlDB is an event streaming database. It does this by applying an SQL interface for event stream processing. Under the covers, ksqlDB uses Kafka Streams to perform its event streaming tasks. A key advantage of ksqlDB is that it *allows you to specify your event streaming operation in SQL*; no code is required.

****

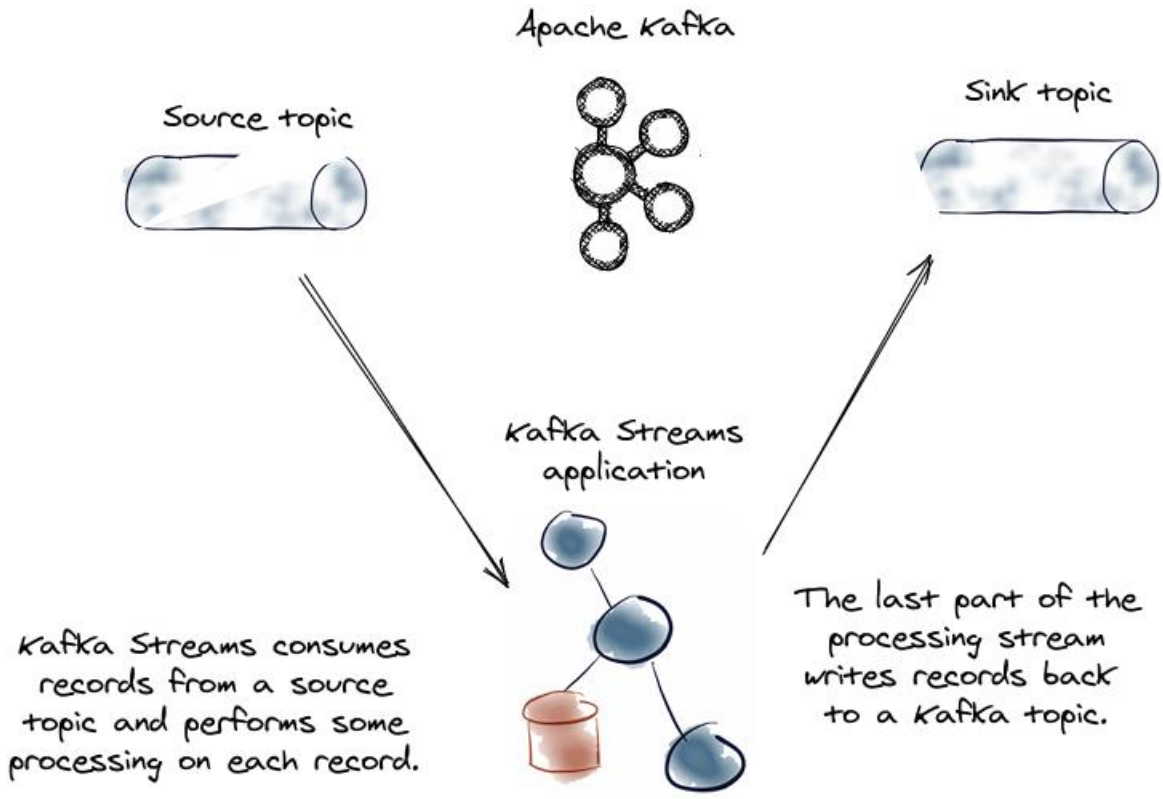

Figure 1.8 Kafka Streams is the stream processing API for Kafka.

****

```
CREATE TABLE activePromotions AS
  SELECT rideId,
         qualifyPromotion(kmToDst) AS promotion
  FROM locations
  GROUP BY rideId
  EMIT CHANGES;


  SELECT rideId, promotion
  FROM activePromotions
  WHERE ROWKEY = '6fd0fcdb';
```

Figure 1.9 ksqlDB provides streaming database capabilities.

****

Note 1: We can call the operation from receiving email that contains coupon to process order is clickstream events.

Note 2: We can use the SMT with security operations, like masking the data of Credit Card, obscure email addresses in logs.

****

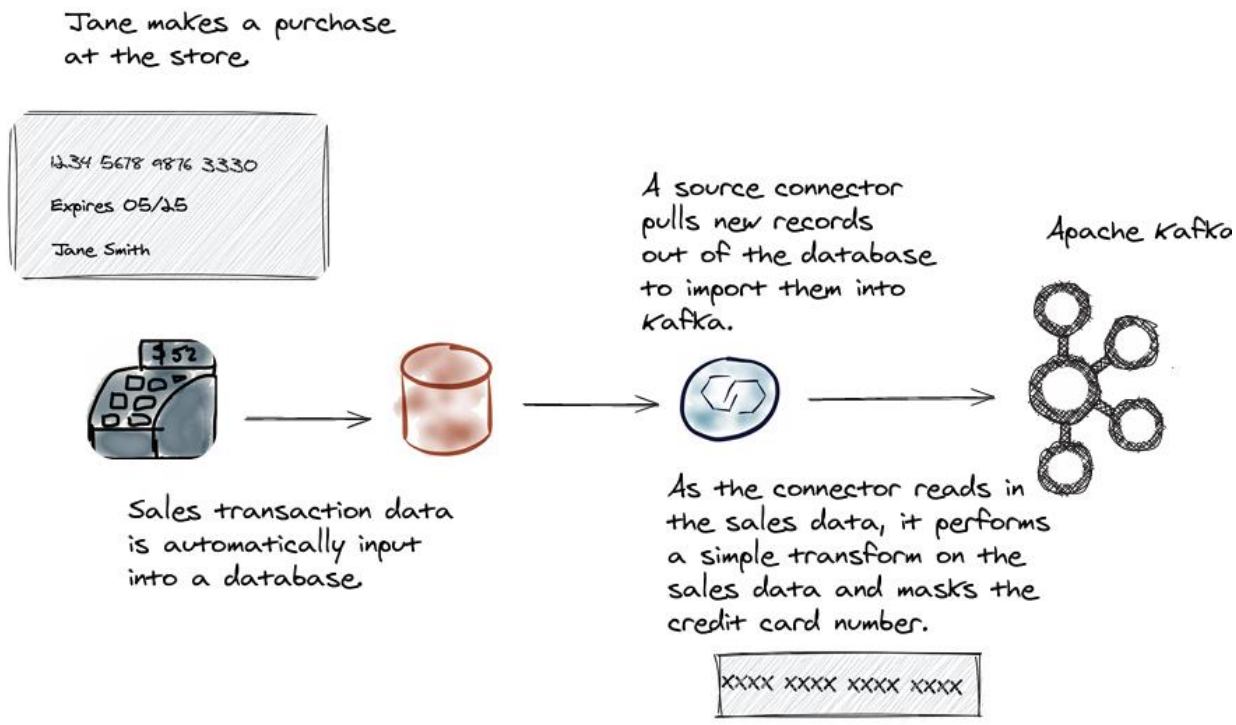

Figure 1.10 Sending all of the sales data directly into Kafka with Connect masking the credit card numbers as part of the process

****

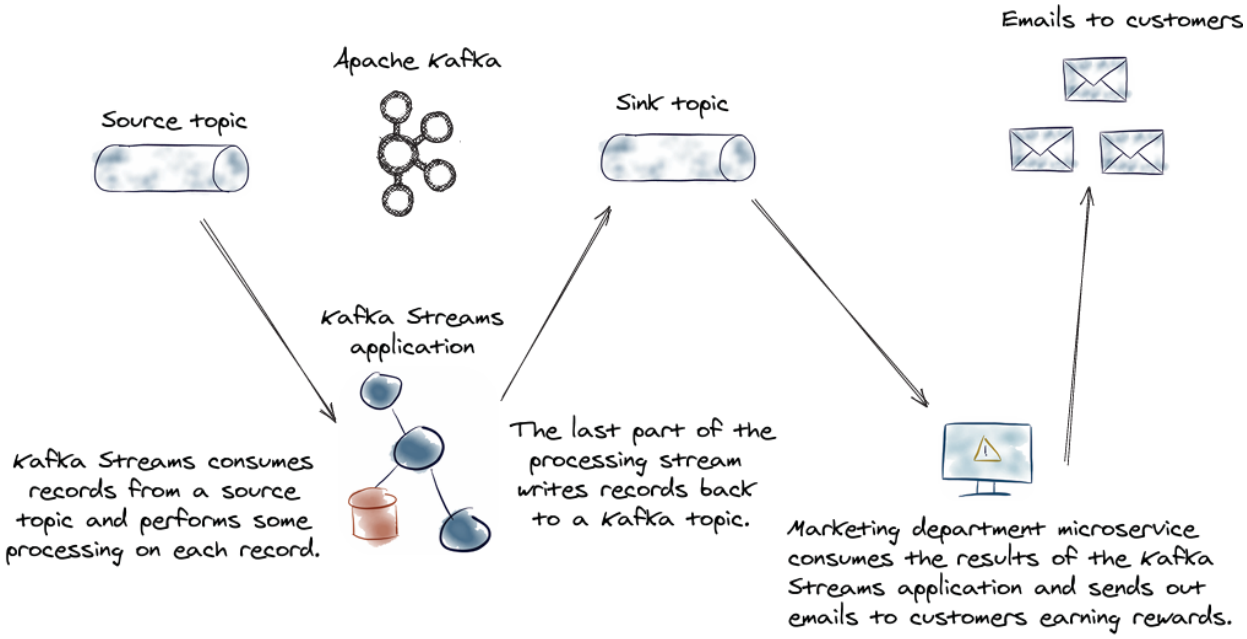

Figure 1.11 Marketing department application for processing customer points and sending out earned emails